سند فنی / نسخه ۱.۰

نرم‌افزار جامع مدیریت تولید و پخش کیک‌پزی

سند معماری فنی، طراحی پایگاه داده (ERD) و نقشه راه اجرایی برای جایگزینی سیستم شکننده فعلی با یک پلتفرم پایدار، چندکاربره و تحت وب.

Django 4.2 | PostgreSQL 16 | Next.js 14 | RTL / Vazirmatn | FEFO / Weighted Avg | Multi-tenant

PREPARED BY
تیم ارشد مهندسی نرم‌افزار

DATE
۲۰۲۵ / ۱۴۰۴

CLASSIFICATION
سند معماری

EXECUTIVE SUMMARY

خلاصه اجرایی

این سند معماری برای طراحی و پیاده‌سازی یک **نرم‌افزار جامع مدیریت تولید و پخش** برای یک کارگاه کیک‌پزی متوسط‌بزرگ تهیه شده است. هدف اصلی پروژه جایگزینی یک فایل نرم‌افزاری شکننده فعلی با یک سیستم پایدار، چندکاربره و تحت وب است که بتواند منطق پیچیده تولید، انبار، فروش و حسابداری را در یک پلتفرم یکپارچه مدیریت کند.

کسب‌وکار با تولید بیش از ۳۰ نوع کیک با تاریخ انقضای کوتاه (حدود ۱۵ روز)، پخش به ۲۰۰ فروشگاه فیزیکی و چند پلتفرم آنلاین از طریق ۳ عامل پخش، و مدل فروش ترکیبی (امانی و قطعی) کار می‌کند. این پیچیدگی نیازمند ردیابی دقیق بچ (Batch) و لات (Lot) بر اساس روش FEFO (First-Expired-First-Out) و محاسبه بهای تمام‌شده به روش میانگین موزون است.

۴۱	۸ ماه	۶
جدول پایگاه داده	مدت توسعه	فاز اجرایی

از نظر فنی، سیستم با **Django 4.2 + Django REST Framework** در بک‌اند، **PostgreSQL 16** به‌عنوان پایگاه داده مرکزی، و **Next.js 14 + TypeScript + Tailwind RTL** در فرانت‌اند پیاده‌سازی می‌شود. Redis برای کش و صف Celery برای کارهای پس‌زمینه (ارسال پیامک، گزارش‌گیری، هشدار انقضا) استفاده خواهد شد. تمام ۴۱ جدول کسب‌وکار در یک پایگاه داده مرکزی با پشتیبانی کامل از ACID و تراکنش‌های دوطرفه طراحی شده‌اند.

نقشه راه توسعه شش‌فازی در هشت ماه شامل: (۱) بن‌بست و مدل پایه، (۲) تولید و انبار، (۳) فروش و پخش، (۴) حسابداری و گزارش، (۵) موبایل و اعلان، و (۶) تست و بهینه‌سازی است. این فازبندی به تیم اجازه می‌دهد در هر مرحله یک MVP قابل دمو تحویل دهد و در عین حال ریسک‌های فنی را در همان مراحل اولیه شناسایی کند.

مزایای کلیدی

سیستم پیشنهادی به‌طور خودکار اسناد حسابداری دوطرفه را به محض ثبت فروش یا خرید تولید می‌کند، کالاهای نزدیک به انقضا را شناسایی و انتقال از فروشگاه‌های کم‌فروش به پرفروش را پیشنهاد می‌دهد، و با ارسال پیامک خودکار، یادآوری سرکشی به عوامل پخش را مدیریت می‌کند. این مزایا در یک پلتفرم با پشتیبانی کامل از RTL، تقویم شمسی و دسترسی چندکاربره همگرا شده‌اند.

معرفی و تحلیل کسب‌وکار

درک دامنه، چالش‌ها و الزامات فنی پروژه

۱.۱ زمینه و چالش فعلی

کارگاه کیک‌پزی در حال حاضر از یک فایل نرم‌افزاری شکننده (احتمالاً یک فایل اکسل پیچیده با ماکرو یا یک برنامه دسکتاپ تک‌کاربره قدیمی) برای مدیریت عملیات روزانه استفاده می‌کند. این سیستم با رشد کسب‌وکار در طول چند سال گذشته به طور جدی تضعیف شده است: پشتیبانی همزمان چند کاربر را ندارد، گزارش‌گیری بلادرنگ را پشتیبانی نمی‌کند، در محاسبه بهای تمام‌شده دچار خطاهای دستی می‌شود، و هیچ کنترلی برای ردیابی دقیق لات و تاریخ انقضا ارائه نمی‌دهد.

این محدودیت‌ها به طور مستقیم بر تصمیم‌گیری‌های مدیریتی و عملکرد مالی کسب‌وکار تأثیر می‌گذارند. عدم ردیابی دقیق FEFO باعث افزایش ضایعات و کالاهای منقضی می‌شود، نبود یکپارچگی بین انبار و حسابداری خطاهای ثبت را افزایش می‌دهد، و عدم دسترسی موبایلی برای عوامل پخش، فرایند سرکشی را کند و خطاپذیر می‌سازد. این محدودیت‌ها به طور روزافزون با رشد کسب‌وکار تشدید می‌شوند و در نهایت به یک گلوگاه عملیاتی تبدیل خواهند شد.

۱.۲ مشخصات کسب‌وکار

کسب‌وکار در یک محیط عملیاتی با مشخصات زیر فعالیت می‌کند که هر کدام الزامات طراحی خاص خود را بر معماری سیستم تحمیل می‌کنند:

- **تنوع محصول:** بیش از ۳۰ نوع کیک با دسته‌بندی‌های مختلف (کرمی، شکلاتی، میوه‌ای، ویژه مناسبتی). هر محصول دارای دستور تولید خاص، قیمت تمام‌شده و چرخه انقضای مستقل است. تنوع بالا نیاز به یک سیستم پویا برای تعریف محصول با ویژگی‌های سفارشی دارد.
- **تاریخ انقضای کوتاه:** هر کیک بین ۱۲ تا ۱۵ روز ماندگاری دارد که مدیریت FEFO را به یک الزام حیاتی تبدیل می‌کند. هر روز تأخیر در تصمیم‌گیری انتقال یا فروش، یک روز از چرخه فروش مفید را کم می‌کند و احتمال ضایعات را افزایش می‌دهد.
- **شبکه توزیع گسترده:** ۲۰۰ فروشگاه فیزیکی فعال در چند استان و چند پلتفرم آنلاین (اینستاگرام، سایت اختصاصی، اسنپ‌فود) که از طریق ۳ عامل پخش مستقل با مسیرهای متفاوت سرو می‌شوند. هر عامل روزانه ۳۰ تا ۵۰ فروشگاه را سرکشی می‌کند.
- **مدل فروش ترکیبی:** بخشی از فروش امانی (Consignment) است که در آن کالا تا فروش نهایی به فروشگاه منتقل نشده و مالکیت در دست کارگاه می‌ماند، و بخشی قطعی (Firm Sale) است که در آن فروش مستقیم به فروشگاه انجام می‌شود. این دو مدل نیاز به منطق متفاوت در انبار و حسابداری دارند.
- **الزام حسابداری:** نیاز به محاسبه دقیق بهای تمام‌شده با روش میانگین موزون، ردیابی بچ/لات، و یکپارچگی کامل با حسابداری دوطرفه برای تحلیل سود و زیان دقیق در هر سطح محصول، فروشگاه و عامل.

۱.۳ اهداف فنی

نرم‌افزار جدید باید اهداف فنی زیر را محقق سازد تا الزامات کسب‌وکار را برآورده کند و در عین حال یک پلتفرم پایدار و قابل توسعه برای سال‌های آینده باشد. این اهداف شاخص‌های قابل اندازه‌گیری دارند که در طول پروژه رصد خواهند شد:

هدف فنی	شرح	معیار موفقیت
چندکاربره	پشتیبانی همزمان ۵۰+ کاربر با نقش‌های متفاوت (مدیر، حسابدار، انباردار، عامل) بدون قفل‌گذاری رکورد.	Response time < 300ms در P95، Uptime ≥ ۹۹.۵٪
تحت وب	دسترسی از هر مرورگری بدون نیاز به نصب، پشتیبانی کامل از مرورگرهای موبایل و دسکتاپ.	Lighthouse Score > ۹۰ در Performance, Accessibility
راست‌چین و شمسی	تمام رابط‌ها به فارسی RTL با تقویم جلالی، اعداد فارسی و قالب‌بندی صحیح تاریخ/زمان.	۱۰۰٪ صفحات با dir="rtl"، تقویم شمسی در همه Date Pickerها
یکپارچگی انبار-حسابداری	ثبت خودکار اسناد حسابداری به محض ثبت هر سند انبار (فروش، خرید، انتقال، برگشت) با کنترل توازن.	۰ خطای توازن بدهکار/بستانکار در پایان روز
پایداری و توسعه‌پذیری	معماری ماژولار با جداسازی وابستگی‌ها، تست‌پذیری بالا، مستندسازی API.	Pylint Score > ۸، Code Coverage > ۸۰٪

۱.۴ فرضیات و محدودیت‌ها

این سند با در نظر گرفتن فرضیات زیر تنظیم شده است. هرگونه تغییر در این فرضیات نیازمند بازنگری در معماری و نقشه راه است و باید در فاز صفر پروژه به‌صورت رسمی تأیید شود:

- تیم توسعه شامل ۱ معمار/توسعه‌دهنده ارشد بک‌اند، ۲ توسعه‌دهنده Mid-level، ۱ توسعه‌دهنده فرانت‌اند و ۱ QA Tester است که به‌صورت تمام‌وقت روی پروژه کار می‌کنند.
- زیرساخت میزبانی روی یک VPS اختصاصی یا کلاد (هلست‌ایران یا تهران‌سرور) با حداقل ۱۶GB RAM و ۴ هسته CPU انجام می‌شود، با امکان ارتقاء به ۳۲GB RAM در فاز تولید.
- هیچ انتقال داده‌ای از سیستم قدیمی به‌صورت خودکار انجام نمی‌شود؛ تمام داده‌های اولیه (محصولات، مشتریان، موجودی افتتاحیه) به‌صورت دستی وارد می‌شوند. دلیل: ناپایداری سیستم قدیمی و عدم وجود ساختار استاندارد داده.
- سرویس پیامک از طریق پروایدرهای ایرانی (کاوه‌نگار یا فراپیامک) تأمین می‌شود با Reserve Provider دوم برای اطمینان از پایداری.
- پروژه به‌صورت داخلی (On-Premise یا VPS داخل ایران) میزبانی می‌شود تا با مقررات داده‌های ایران سازگار باشد و دسترسی سریع برای کاربران داخلی فراهم شود.
- زبان پیش‌فرض سیستم فارسی است؛ انگلیسی فقط در گزارش‌های فنی، لاگ‌ها و مستندات API استفاده می‌شود.

معماری فنی (C4 Model)

لایه‌بندی سیستم، انتخاب فناوری و نمودار C4 Level 1 و 2

۲.۱ نمای کلی معماری

معماری پیشنهادی یک مدل **Layered Monolith** است که ترکیبی از سادگی استقرار یک Monoolith و نظم یک معماری لایه‌ای تمیز است. این انتخاب بر اساس سه معیار کلیدی انجام شده: (۱) اندازه تیم کوچک (۴-۵ نفر)، (۲) نیاز به تحویل سریع MVP در ۸ ماه، و (۳) نیاز به پایداری بالا بدون پیچیدگی‌های توزیعی Microservice. در صورت رشد آتی کسب‌وکار، تمایل ماژولار Django اجازه استخراج سرویس‌ها را در آینده فراهم می‌کند.

سیستم از شش لایه اصلی تشکیل شده که هر کدام مسئولیت مشخصی دارند و از طریق رابط‌های قراردادی (API contracts) با هم ارتباط برقرار می‌کنند. این جداسازی باعث می‌شود تغییر در یک لایه (مثلاً تعویض پروایدر پیامک) نیازی به تغییر در لایه‌های دیگر نداشته باشد. تمام ارتباطات بین لایه‌های داخلی از طریق Dependency Injection و رابط‌های (Interfaces) انجام می‌شود که تست‌پذیری و انعطاف‌پذیری را بالا می‌برد.



شکل ۱: لایه‌بندی سیستم از کلاینت تا یکپارچگی‌های خارجی

۲.۲ انتخاب فناوری و обоснованان

انتخاب فناوری برای هر لایه بر اساس سه معیار انجام شده: (۱) بلوغ و پایداری اکوسیستم، (۲) در دسترس بودن منابع انسانی در بازار ایران، و (۳) سازگاری با الزامات خاص پروژه (RTL، شمسی، FEFO، حسابداری دوطرفه). جدول زیر обоснованان انتخاب‌ها را به‌طور شفاف شرح می‌دهد:

لایه	فناوری انتخابی	دلیل انتخاب
بک‌اند	Django 4.2 + DRF	ORM قدرتمند، Admin آماده، امنیت داخلی، اکوسیستم پایتون برای حسابداری و محاسبات عددی با کتابخانه‌هایی مثل NumPy و Pandas، سازگاری با Celery.

لایه	فناوری انتخابی	دلیل انتخاب
پایگاه داده	PostgreSQL 16	پشتیبانی ACID کامل، JSONB برای فرم‌های پویا، عملکرد بالا روی Volume داده متوسط (۲۰۰ فروشگاه × ۳۰ محصول × ۳۶۵ روز ≈ ۲.۲M رکورد در سال)، Trigger Functions برای کنترل توازن حسابداری، Open-source و بدون لایسنس.
فرانت‌اند	Next.js 14 + TypeScript	SSR برای SEO و عملکرد اولیه، TypeScript برای امنیت نوع در منطق پیچیده، اکوسیستم Tailwind RTL قدرتمند، PWA برای دسترسی آفلاین عوامل.
کش و صف	Redis 7 + RabbitMQ 3	Redis برای کش کوئری‌های پرتکرار (موجودی لحظه‌ای، لیست محصولات) و Session، RabbitMQ به‌عنوان Broker پایدارتر برای Celery در حجم بالا.
Async Workers	Celery 5 + Beat	ارسال پیامک‌های زمان‌بندی‌شده، تولید گزارش‌های سنگین در پس‌زمینه، اجرای شبانه Job هشدار FEFO، مانیتورینگ توازن حسابداری روزانه.
تقویم و RTL	jalaali-js + Vazirmatn	jalaali-js استاندارد بازار ایران برای تبدیل میلادی-شمسی، Vazirmatn فونت فارسی مدرن با وزن‌های کامل (۳۰۰-۹۰۰) و پشتیبانی اعداد فارسی.
Deployment	Docker Compose + Nginx	Docker برای سادگی استقرار و Nginx، rollback به‌عنوان Reverse Proxy با SSL و Rate Limiting.
مانیتورینگ	Sentry + Grafana	Sentry برای خطاهای کد (Frontend و Backend)، + Grafana Prometheus برای Metrics زیرساختی.

۲.۳ جریان داده و تراکنش‌ها

در معماری پیشنهادی، تمام درخواست‌های کاربر از طریق Nginx وارد لایه API Django می‌شوند. لایه API پس از احراز هویت (JWT) و کنترل نقش (RBAC)، درخواست را به Service مربوطه ارجاع می‌دهد. Serviceها منطق کسب‌وکار را در یک تراکنش پایگاه داده اجرا می‌کنند و در پایان، در صورت تغییر داده‌های انبار یا مالی، یک سیگنال (Signal) Django ارسال می‌کنند که توسط Accounting Service و Notification Service دریافت شده و اقدامات لازم (ثبت سند حسابداری، ارسال پیامک) را به‌صورت همزمان یا ناهمزمان انجام می‌دهد.

این الگو (Signal-Driven) جداسازی واضحی بین لایه‌ها فراهم می‌آورد: هیچ Serviceای مستقیماً جدول حسابداری را دستکاری نمی‌کند. در عین حال، تمام تراکنش‌های بحرانی (مثل ثبت فروش) از یک Database Transaction استفاده می‌کنند که در صورت بروز هر خطایی، تمام اثرات rollback می‌شوند. این تضمین می‌کند که هم انبار و هم حسابداری همواره همگام هستند.

> **نکته طراحی**
>
> استفاده از Django Signals برای یکپارچگی انبار-حسابداری سادگی توسعه را بالا می‌برد، اما در حجم بالا ممکن است به یک گلوگاه تبدیل شود. در فاز ۶ (تست و بهینه‌سازی) پیشنهاد می‌شود این ارتباط به Celery Task ناهمزمان منتقل شود تا تراکنش اصلی فروش سریع‌تر commit شود و سند حسابداری در پس‌زمینه تولید گردد.

طراحی ماژولار بک‌اند (Django Apps)

جداسازی Domainها، الگوی Service-Repository و کنترل دسترسی RBAC

۳.۱ ساختار Apps

بک‌اند Django به ۱۱ ماژول مستقل (App) تقسیم می‌شود که هر کدام یک Domain مشخص کسب‌وکار را پوشش می‌دهند. این ماژولار بودن دو فایده اصلی دارد: (۱) جداسازی واضح مسئولیت‌ها که تیم توسعه می‌تواند هم‌زمان روی چند ماژول کار کند، و (۲) امکان استخراج آینده به Microservice در صورت رشد شدید حجم. هر App دارای مدل‌های مختصر به خود، Service Layer برای منطق کسب‌وکار، و Repository Pattern برای دسترسی به داده است.

App	نام فارسی	مسئولیت	مدل‌های اصلی
`accounts`	کاربران و نقش‌ها	احراز هویت JWT، مدیریت نقش‌ها (RBAC)، پروفایل کاربر، لاگ فعالیت.	User, Role, Permission, UserProfile, ActivityLog
`products`	محصولات و دسته‌بندی	تعریف محصول، دسته‌بندی، واحد، فرمول تولید، قیمت‌گذاری.	Product, ProductCategory, Unit, BOM, Price
`inventory`	انبار و موجودی	موجودی لات/بچ، حرکت‌های انبار، موجودی لحظه‌ای، FEFO Engine.	InventoryLot, InventoryMovement, StockBalance, Warehouse
`production`	تولید و مصرف	ثبت بچ تولید، مصرف مواد اولیه، خروج محصول نهایی، محاسبه راندمان.	ProductionBatch, ProductionConsumption, ProductionOutput
`dispatch`	پخش و بارگیری	سند پخش به عامل، بارگیری و انتقال بین انبار و فروشگاه، ردیابی مسیر عامل.	DispatchHeader, DispatchLine, Route, Vehicle
`sales`	فروش و برگشت	سند فروش امانی/قطعی، سند برگشت از فروشگاه، تسویه فروش امانی.	SalesHeader, SalesLine, ReturnHeader, ReturnLine
`transfers`	انتقال بین فروشگاه‌ها	انتقال کالا بین دو فروشگاه، پیشنهاد انتقال FEFO، تأیید دوطرفه.	TransferHeader, TransferLine, TransferSuggestion
`partners`	فروشگاه‌ها و عوامل	مدیریت ۲۰۰ فروشگاه، ۳ عامل پخش، مشتریان آنلاین، اعتبار هر فروشگاه.	Store, Distributor, Customer, PartnerCredit
`accounting`	حسابداری دوطرفه	سند حسابداری، رشد حساب‌ها (کدینگ)، کنترل توازن بدهکار/بستانکار، تراز.	Account, JournalHeader, JournalLine, FiscalPeriod
`notifications`	اعلان و پیامک	صف پیامک، قالب پیام، تاریخچه ارسال، زمان‌بندی یادآوری سرکشی.	SmsTemplate, SmsQueue, SmsLog, NotificationSchedule

App	نام فارسی	مسئولیت	مدل‌های اصلی
`reports`	گزارش و داشبورد	گزارش فروش روزانه/ماهانه، ضایعات، سود و زیان، داشبورد بلادرنگ.	ReportDefinition, SavedReport, DashboardWidget

۳.۲ الگوی Service-Repository

برای حفظ جداسازی لایه‌ها، هر App از سه لایه داخلی تشکیل شده است. لایه **View/Serializer** مسئول اعتبارسنجی ورودی و تبدیل داده به JSON است. لایه **Service** منطق کسب‌وکار را در خود جای داده و تمام تراکنش‌های DB را مدیریت می‌کند. لایه **Repository** دسترسی به داده‌ها را کپسوله می‌کند و اجازه می‌دهد در آینده منبع داده (مثلاً برای Migration به Microservice) بدون تأثیر بر Service تغییر کند.

```
# ساختار پوشه‌بندی هر App
products/
├── models.py          # مدل‌های DB
├── repositories/
│   ├── product_repo.py
│   └── category_repo.py
├── services/
│   ├── product_service.py     # منطق کسب‌وکار
│   ├── price_service.py
│   └── bom_service.py         # فرمول تولید
├── serializers/
│   ├── product_serializer.py
│   └── category_serializer.py
├── views/
│   ├── product_viewset.py     # API endpoints
│   └── category_viewset.py
├── signals.py          # Signal handlers (یکپارچگی)
├── tasks.py            # Celery tasks (Async)
├── selectors.py        # کوئری‌های پیچیده Read
└── tests/
    ├── test_product_service.py
    └── test_product_api.py
```

۳.۳ کنترل دسترسی RBAC

سیستم نقش‌ها با ۴ نقش اصلی طراحی شده است که هر کدام مجموعه مشخصی از Permissionها دارند. این نقش‌ها در جدول `Role` تعریف و در `Permission` به‌صورت دقیق (per-action: view, add, change, delete, approve) کنترل می‌شوند. پیاده‌سازی از طریق یک Middleware سفارشی و DRF Permission Class انجام می‌شود که در هر درخواست، نقش کاربر و Permission مورد نیاز را بررسی می‌کند.

نقش	دسترسی‌های کلیدی	محدودیت‌ها
مدیر	تمام گزارش‌ها، تأیید انتقال FEFO، تنظیمات سیستم، تعریف محصول و کاربر.	دسترسی به اسناد حسابداری فقط در حالت Read. عدم امکان ویرایش سند ثبت‌شده (فقط معکوس).
حسابدار	ثبت سند دستی، کنترل توازن، تسویه روزانه، گزارش‌های مالی.	عدم دسترسی به تعریف محصول، عدم تغییر در موجودی انبار (فقط مشاهده).
انباردار	ثبت تولید، ثبت ورود/خروج کالا، تأیید انتقال بین انبارها.	عدم دسترسی به قیمت‌گذاری، عدم مشاهده سود و زیان، عدم دسترسی به کاربران.
عامل پخش	مشاهده سبد پخش روزانه، ثبت فروش امانی/قطعی، ثبت برگشت، مشاهده موجودی فروشگاه.	عدم دسترسی به بهای تمام‌شده، عدم تغییر قیمت، فقط فروشگاه‌های اختصاص‌یافته.

پیاده‌سازی امنیتی

علاوه بر RBAC، تمام APIها از Rate Limiting (۱۲۰ درخواست در دقیقه برای هر کاربر) و JWT با Refresh Token کوتاه (۱۵ دقیقه) و Long Token (۷ روز) استفاده می‌کنند. تمام عملیات حساس (حذف، تأیید انتقال، ثبت سند حسابداری) با Soft Delete و Activity Log همراه است تا هیچ داده‌ای به‌طور فیزیکی حذف نشود و هر تغییر قابل ردیابی باشد.

SECTION 04

طراحی پایگاه داده و ERD

۴۱ جدول در ۵ گروه منطقی با روابط کلیدی و کنترل‌های یکپارچگی

۴.۱ نمای کلی ۴۱ جدول

پایگاه داده مرکزی شامل ۴۱ جدول است که در ۵ گروه منطقی دسته‌بندی شده‌اند. این دسته‌بندی بر اساس Domain کسب‌وکار انجام شده و با ساختار Appهای Django (بخش ۳) هماهنگ است. تمام جداول از کلید اصلی id (BIGINT AUTO_INCREMENT)، فیلدهای ردیابی created_by، updated_at، created_at و در صورت نیاز deleted_at (Soft Delete) استفاده می‌کنند.

۶
جدول پایه

۷
انبار و تولید

۱۲
فروش و پخش

۴.۲ گروه ۱ — جداول پایه (۶ جدول)

این جداول هسته داده‌های اصلی سیستم را نگه‌داری می‌کنند و به‌ندرت تغییر می‌کنند. تمام جداول دیگر به این جداول ارجاع دارند. طراحی این جداول باید در فاز ۱ پروژه کاملاً تثبیت شود زیرا تغییر در آن‌ها در فازهای بعد به‌سختی انجام می‌شود.

Products (محصولات)

ProductCategories (دسته‌بندی)

Stores (فروشگاه‌ها)

Distributors (عوامل پخش)

Customers (مشتریان آنلاین)

Accounts (کدینگ حساب‌ها)

جدول	فیلدهای کلیدی	توضیح
Products	id, code, name, category_id, unit_id, shelf_life_days, default_price, is_active	۳۰+ محصول با کد یکتا، مدت ماندگاری، قیمت پایه. shelf_life_days برای FEFO.
ProductCategories	id, name, parent_id, level	ساختار درختی (Self-Reference) با ۳ سطح: کیک کرمی → کیک کرمی کوچک.
Stores	id, code, name, address, phone, geo_lat, geo_lng, distributor_id, sales_model, is_active	۲۰۰ فروشگاه با مختصات GPS و مدل فروش (امانی/قطعی/ترکیبی).
Distributors	id, code, name, mobile, vehicle_id, default_route, commission_rate	۳ عامل با مسیر پیش‌فرض و درصد پورسانت.
Customers	id, type, name, phone, platform (instagram/snapp/website), credit_limit	مشتریان آنلاین با سقف اعتبار.
Accounts	id, code (یکتا), name, type (asset/liability/equity/revenue/expense), parent_id, is_active	کدینگ درختی حساب‌ها با ۵ نوع اصلی. مثلاً: 1010 → موجودی کالا، 1011 → کیک کرمی.

۴.۳ گروه ۲ — انبار و تولید (۷ جدول)

این گروه قلب منطق FEFO و میانگین موزون است. هر موجودی در سیستم به‌صورت **Lots** ردیابی می‌شود که یک لات معادل یک بچ تولید در یک تاریخ خاص است. تمام حرکت‌ها (ورود، خروج، انتقال، برگشت) در جدول InventoryMovements ثبت و موجودی لحظه‌ای در InventoryLots نگه‌داری می‌شود.

InventoryLots (لات‌های انبار)

InventoryMovements (حرکت‌ها)

ProductionBatches (بچ تولید)

ProductionConsumption (مصرف مواد)

ProductionOutputs (خروج محصول)

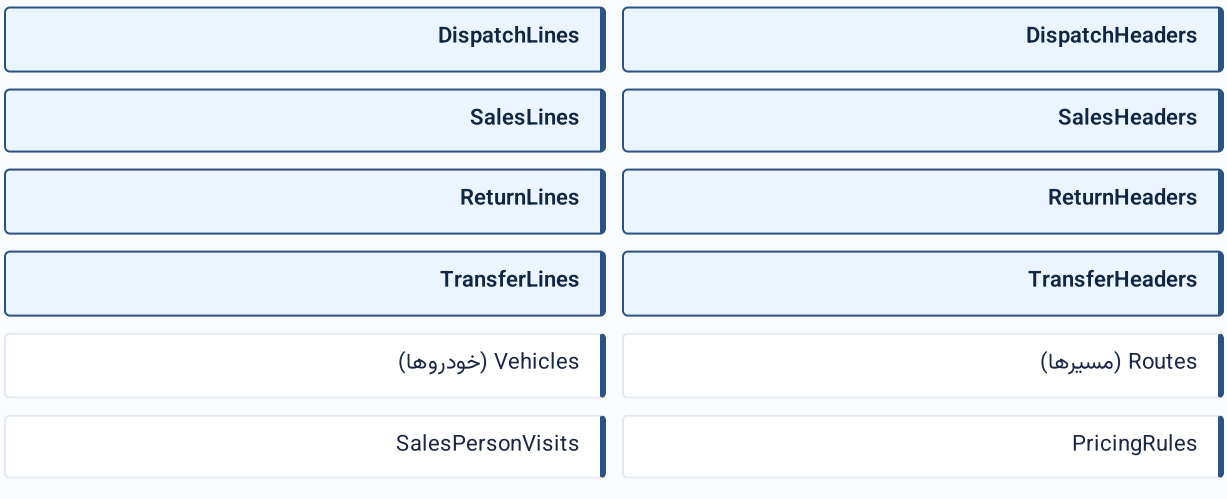
Warehouses (انبارها)

StockBalances (موجودی لحظه‌ای)

جدول	فیلدهای کلیدی	توضیح
InventoryLots	id, product_id, batch_no, production_date, expiry_date, initial_qty, current_qty, weighted_avg_cost, warehouse_id, status	هر لات معادل یک بچ تولید. expiry_date برای FEFO. weighted_avg_cost میانگین موزون.
InventoryMovements	id, lot_id, movement_type (in/out/transfer/return), qty, ref_doc_type, ref_doc_id, warehouse_id, created_at	تاریخچه کامل حرکت با ارجاع به سند مبدأ (فروش، تولید، انتقال).
ProductionBatches	id, batch_no, product_id, planned_qty, actual_qty, start_at, end_at, status, weighted_avg_cost	هر بار تولید یک بچ با مقدار برنامه‌ریزی و واقعی و محاسبه بهای تمام‌شده.
ProductionConsumption	id, batch_id, material_lot_id, qty_consumed, cost_at_consumption	مواد اولیه مصرفی برای هر بچ با هزینه لحظه‌ای.
ProductionOutputs	id, batch_id, product_id, qty_produced, qty_waste, lot_id (به InventoryLots)	خروج محصول نهایی با محاسبه ضایعات و تولید لات جدید.
Warehouses	id, code, name, type (main/store/virtual), store_id (nullable), is_active	انبار اصلی کارگاه + ۲۰۰ انبار مجازی هر فروشگاه (امانی).
StockBalances	id, product_id, warehouse_id, total_qty, total_value, last_lot_id, updated_at	موجودی تجمیعی هر محصول در هر انبار (Cache برای عملکرد بالا).

۴.۴ گروه ۳ – فروش و پخش (۱۲ جدول)

گروه فروش شامل تمام اسناد مرتبط با چرخه پخش و فروش است. هر سند از ساختار Header-Line استفاده می‌کند که در آن Header اطلاعات کلی سند (شماره، تاریخ، عامل، فروشگاه) و Lineها اقلام جزئی (محصول، تعداد، قیمت، لات) را نگه‌داری می‌کنند. این الگو اجازه می‌دهد یک سند چندین محصول را در یک تراکنش ثبت کند.



جدول	فیلدهای کلیدی	توضیح
DispatchHeaders	id, dispatch_no, distributor_id, dispatch_date, total_qty, status, vehicle_id	سند پخش به عامل با مجموع اقلام و وضعیت (pending/in_transit/arrived).
DispatchLines	id, dispatch_id, product_id, lot_id, qty, dest_store_id	اقلام پخش با مقصد هر فروشگاه.
SalesHeaders	id, sale_no, store_id, sale_date, sales_type (consignment/firm), total_amount, total_cost, status, journal_id	سند فروش با نوع امانی یا قطعی و ارجاع به سند حسابداری.
SalesLines	id, sale_id, product_id, lot_id, qty, unit_price, total_price, unit_cost, line_total	اقلام فروش با لات مشخص (برای FEFO) و بهای تمام‌شده لحظه‌ای.
ReturnHeaders	id, return_no, store_id, return_date, original_sale_id, total_qty, reason	برگشت از فروشگاه با علت (منقضی/م شکسته/قابل فروش).
ReturnLines	id, return_id, product_id, lot_id, qty, condition (sellable/expired/damaged)	اقلام برگشت با وضعیت که تعیین می‌کند کالا به انبار برمی‌گردد یا ضایعات.
TransferHeaders	id, transfer_no, from_store, to_store, transfer_date, status, initiator_id	انتقال بین دو فروشگاه با تأیید دوطرفه (initiator و receiver).
TransferLines	id, transfer_id, product_id, lot_id, qty, expiry_date	اقلام انتقال با لات و تاریخ انقضا برای تأیید FEFO.
Routes	id, distributor_id, day_of_week, store_sequence (JSON)	مسیر هر عامل در هر روز هفته با ترتیب فروشگاه‌ها.
Vehicles	id, plate, capacity, driver_name, fuel_card	خودروهای پخش با ظرفیت.
PricingRules	id, product_id, store_tier (A/B/C), min_price, default_price, effective_date	قیمت‌گذاری پویا بر اساس لایه فروشگاه (پرفروش/A، متوسط/B، کم‌فروش/C).
SalesPersonVisits	id, distributor_id, store_id, visit_date, check_in_time, check_out_time, sales_count	ردیابی سرکشی روزانه عامل با GPS و زمان ورود/خروج.

۴.۵ گروه ۴ – حسابداری (۶ جدول)

گروه حسابداری از منطق Double-Entry استاندارد پیروی می‌کند. هر سند حسابداری (JournalHeader) چندین خط (JournalLine) دارد که مجموع بدهکارها باید با مجموع بستانکارها برابر باشد. این تضمین توسط

یک **Database Trigger** در PostgreSQL کنترل می‌شود تا در صورتی که یک تراکنش تلاش کند سند نامتوازن ثبت کند، کل تراکنش Reject شود.

JournalHeaders	JournalLines
Accounts	FiscalPeriods
CostCenters	JournalTemplates

جدول	فیلدهای کلیدی	توضیح
`JournalHeaders`	id, journal_no, date, source_type (sale/purchase/transfer/manual), source_id, total_debit, total_credit, status, period_id	سند حسابداری با نوع منبع تولیدکننده و کنترل توازن در DB Trigger.
`JournalLines`	id, journal_id, account_id, debit, credit, cost_center_id, description, ref_doc_id	خطوط سند با حساب، مبلغ بدهکار و بستانکار.
`FiscalPeriods`	id, name, start_date, end_date, is_closed, closed_at	دوره‌های مالی (ماهانه/سالانه) با امکان بستن دوره و جلوگیری از ویرایش.
`CostCenters`	id, code, name, parent_id, type	مرکز هزینه (خط تولید، دپارتمان، انبار) برای تحلیل سود/زیان دقیق.
`JournalTemplates`	id, name, source_type, template_json (حساب‌های بدهکار و بستانکار پیش‌فرض)	قالب‌های سند خودکار: فروش → موجودی کالا بستانکار، درآمد فروش بدهکار.
`ExchangeRates`	id, currency, rate, effective_date	نرخ ارز برای گزارش‌های مالی چندارزی (در صورت صادرات).

۴.۶ گروه ۵ — جداول سیستمی (۱۰ جدول)

این جداول توسط خود Django و کتابخانه‌های جانبی استفاده می‌شوند. برخی از این جداول (User, Permission, Group) توسط Django Auth ارائه می‌شوند و بقی توسط پکیج‌های Celery، DRF و Custom Apps اضافه می‌گردند.

Users (Django Auth)	Groups (Django Auth)
Permissions (Django)	UserProfiles
ActivityLogs	CeleryTaskMeta
CeleryTaskResults	DjangoSessions
DjangoMigrations	DjangoAdminLog

۴.۷ روابط کلیدی ERD

روابط بین جداول در نمودار ERD پیچیده است (با ۴۱ جدول، بیش از ۶۰ رابطه وجود دارد). در اینجا روابط بحرانی که منطق کسب‌وکار را شکل می‌دهند خلاصه شده‌اند. روابط دیگر از طریق Foreign Keyهای مشخص شده در جداول بالا قابل استنباط هستند:

- **SalesLines ↔ InventoryLots:** یک به چند. هر SalesLine به یک Lot مشخص ارجاع می‌دهد که تضمین می‌کند ردیابی FEFO در سطح اقلام فروش انجام شود.
- **InventoryLots ↔ ProductionBatches:** یک به یک. هر بچ تولید، یک Lot جدید در InventoryLots ایجاد می‌کند.
- **JournalHeaders ↔ SalesHeaders:** یک به یک (یک اختیاری). هر سند فروش پس از commit، یک سند حسابداری خودکار تولید می‌کند.
- **Stores ↔ Warehouses:** یک به یک. هر فروشگاه یک انبار مجازی (Virtual Warehouse) برای مدل امانی دارد.
- **Distributors ↔ Stores:** چند به چند (از طریق جدول Routes). هر عامل چند فروشگاه را سرو می‌کند و هر فروشگاه ممکن است توسط چند عامل در روزهای مختلف سرو شود.
- **Accounts (Self-Reference):** یک به چند (parent_id). ساختار درختی کدینگ حسابداری.
- **SalesHeaders ↔ ReturnHeaders:** چند به یک. هر برگشت به یک فروش اصلی ارجاع می‌دهد.
- **Stores ↔ TransferHeaders (دو بار):** از from_store و to_store. یک سند انتقال بین دو فروشگاه.

SECTION 05

منطق کسب‌وکار: میانگین موزون

محاسبه بهای تمام‌شده با روش Weighted Average Cost

۵.۱ تعریف و دلیل انتخاب

روش میانگین موزون (Weighted Average Cost) برای محاسبه بهای تمام‌شده انتخاب شده زیرا این روش با ماهیت تولید کیک‌پزی هماهنگ است: مواد اولیه (آرد، شکر، کره، تخم‌مرغ) در خریدهای متعدد و با قیمت‌های متغیر وارد انبار می‌شوند و سپس در بچ‌های تولید مصرف می‌شوند. اختصاص هزینه هر خرید به هر بچ خاص (روش FIFO برای مواد) پیچیدگی عملیاتی بالا دارد و روش میانگین موزون با یکپارچه‌سازی هزینه تمام مواد در یک متغیر، این فرایند را ساده می‌کند.

در روش میانگین موزون، هر بار ورود کالا به انبار (چه از طریق خرید و چه تولید)، هزینه متوسط کالا در انبار بر اساس میانگین وزنی موجودی قبلی و ورودی جدید محاسبه می‌شود. این متوسط وزنی به‌عنوان بهای تمام‌شده هر واحد در خروج‌های بعدی (مثل فروش یا مصرف در تولید) استفاده می‌شود. این روش به‌طور خودکار توسط Service Layer پیاده‌سازی می‌شود و نیازی به محاسبه دستی حسابدار ندارد.

۵.۲ فرمول محاسبه

فرمول پایه میانگین موزون پس از هر ورود جدید به انبار به‌صورت زیر است. این فرمول در هر بار `InventoryMovement` از نوع `in` اجرا می‌شود و فیلد `weighted_avg_cost` در `InventoryLots` را به‌روز می‌کند:

> **فرمول اصلی میانگین موزون**
>
> ```
> New Weighted Avg = (Existing Qty × Old Avg) + (New Qty × New Unit Cost)
> / (Existing Qty + New Qty)
> ```

به‌طور مشخص، در سیستم این فرمول در یک **Database Function** در PostgreSQL به‌صورت زیر پیاده‌سازی می‌شود تا اطمینان حاصل شود که در صورت دسترسی همزمان چند کاربر به داده‌های موجودی، محاسبه به‌درستی و به‌صورت اتمیک انجام می‌شود:

```
-- PostgreSQL Function: محاسبه میانگین موزون پس از ورود
CREATE OR REPLACE FUNCTION update_weighted_avg(
    p_lot_id BIGINT,
    p_in_qty NUMERIC(14,3),
    p_in_unit_cost NUMERIC(18,4)
) RETURNS NUMERIC(18,4) AS
$$DECLARE v_existing_qty NUMERIC(14,3); v_old_avg NUMERIC(18,4); v_new_avg NUMERIC
$$ LANGUAGE plpgsql;
```

۵.۳ مثال عددی گام‌به‌گام

برای درک بهتر، یک مثال عددی واقعی از چرخه تولید و فروش کیک کرمی کوچک در طول یک روز را بررسی می‌کنیم. این مثال نشان می‌دهد چگونه فرمول میانگین موزون در عمل رفتار می‌کند و چرا انتخاب این روش برای کسب‌وکار صحیح است:

گام	رویداد	ورودی	قیمت واحد	موجودی	میانگین موزون جدید
۱	موجودی اول دوره	—	—	۱۰۰ عدد	۲۵٬۰۰۰ ریال
۲	ورود بچ تولید (۵۰ عدد)	۵۰	۲۸٬۰۰۰ ریال	۱۵۰ عدد	=(۱۰۰×۲۵ + ۵۰×۲۸)/۱۵۰ = ۲۶٬۰۰۰ ریال
۳	فروش (۳۰ عدد)	—	—	۱۲۰ عدد	۲۶٬۰۰۰ ریال (بدون تغییر)
۴	ورود بچ تولید (۳۰ عدد)	۳۰	۳۰٬۰۰۰ ریال	۱۵۰ عدد	=(۱۲۰×۲۶ + ۳۰×۳۰)/۱۵۰ = ۲۶٬۸۰۰ ریال
۵	فروش (۶۰ عدد)	—	—	۹۰ عدد	۲۶٬۸۰۰ ریال (بدون تغییر)

جدول ۱: مثال عددی محاسبه میانگین موزون در یک روز عملیاتی

همان‌طور که در جدول مشاهده می‌شود، فروش‌ها میانگین موزون را تغییر نمی‌دهند. تنها ورودی‌های جدید (خرید یا تولید) باعث محاسبه میانگین جدید می‌شوند. این ویژگی برای تحلیل سود/زیان به‌همراه FEFO (که در بخش ۶ بررسی می‌شود) اجازه می‌دهد سود هر فروش با دقت بالا محاسبه شود.

۵.۴ تأثیر برگشت کالا و ضایعات

برگشت کالا از فروشگاه به انبار دو حالت دارد: (۱) کالا قابل فروش مجدد است (مثل اضافه فروشگاه که در ظاهر سالم است) که در این حالت با همان میانگین موزون قبلی به `InventoryLots` برمی‌گردد. (۲) کالا منقضی یا آسیب‌دیده است که در این حالت به‌عنوان ضایعات ثبت و میانگین موزون اقلام باقی‌مانده با فرمول معکوس (حذف از محاسبه) به‌روز می‌شود.

> **نکته محاسباتی**
>
> برای ضایعات، روش استاندارد این است که میانگین موزون تغییر نکند (هزینه ضایعات به‌عنوان Cost of Goods Sold ثبت می‌شود). اما اگر بخواهیم هزینه ضایعات را به سایر اقلام منتقل کنیم (روش متناسب‌سازی)، نیاز به یک Job شبانه Celery داریم که موجودی آخر ماه را بر اساس خالص (منهای ضایعات) دوباره میانگین‌گیری کند. این تصمیم باید با حسابدار ارشد کسب‌وکار در فاز ۴ گرفته شود.

SECTION 06

منطق FEFO و ردیابی بچ/لات

الگوریتم First-Expired-First-Out با شبه‌کد و سناریوهای پیچیده

۶.۱ مفهوم FEFO و تفاوت با FIFO

روش FEFO (First-Expired-First-Out) یک نوع از روش FIFO است که در آن معیار اولویت‌بندی لات‌ها **تاریخ انقضا** است نه تاریخ تولید. این روش برای صنایع غذایی و دارویی حیاتی است زیرا تضمین می‌کند کالاهای با مدت ماندگاری کمتر اول خروج کنند، در نتیجه ضایعات به حداقل می‌رسد. در کسب‌وکار کیک‌پزی با انقضای ۱۵ روزه، حتی یک روز تأخیر در تشخیص لات مناسب می‌تواند منجر به افزایش ۱۰٪ ضایعات شود که در مقیاس سالانه هزینه قابل توجهی است.

تفاوت کلیدی FEFO با FIFO در این است که در FIFO ترتیب ورود به انبار تعیین‌کننده خروج است، اما در FEFO ممکن است یک لات دیرتر وارد شده اما با تاریخ انقضای زودتر، اول خروج کند. این حالت در عمل وقتی رخ می‌دهد که تولید یک محصول از دو فرمول متفاوت انجام شده (مثلاً با دو نوع کره با مدت ماندگاری متفاوت) یا وقتی یک بچ تولید با تاریخ انقضای نزدیک به لات‌های موجود اضافه می‌شود.

۶.۲ الگوریتم انتخاب لات در فروش

هر بار که یک فروش ثبت می‌شود، Service فروش لات مناسب را برای هر خط انتخاب می‌کند. الگوریتم انتخاب شامل چند مرحله است که در شبه‌کد زیر نمایش داده شده است. این الگوریتم به‌گونه‌ای طراحی شده که در صورت نبود لات با انقضای کافی، فروش به‌صورت خودکار Block شود و عامل پخش پیام هشدار دریافت کند:

```
# services/sales_service.py
def allocate_lots_for_sale(
    product_id: int,
    warehouse_id: int,
    qty_needed: Decimal,
    sale_date: date,
) → list[AllocationItem]:
    """
    انتخاب لات‌ها بر اساس FEFO برای یک خط فروش.

    Returns:
        list of (lot_id, qty_from_lot) pairs
    Raises:
        InsufficientStockError: اگر موجودی کافی نباشد
        NearExpiryWarning: اگر نزدیک‌ترین انقضا کمتر از ۳ روز باشد
    """
    # ۱. دریافت تمام لات‌های فعال این محصول در این انبار،
    #    مرتب شده بر اساس تاریخ انقضا (صعودی)
    lots = InventoryLot.objects.filter(
        product_id=product_id,
        warehouse_id=warehouse_id,
        current_qty__gt=0,
        status='active',
        expiry_date__gt=sale_date,  # فقط لات‌های منقضی‌نشده
    ).order_by('expiry_date', 'production_date').select_for_update()

    if not lots.exists():
        raise InsufficientStockError(
            f"موجودی برای محصول {product_id} در انبار {warehouse_id} نیست"
        )

    # ۲. هشدار انقضای نزدیک (۳ روز یا کمتر)
    nearest_expiry = lots.first().expiry_date
    days_to_expiry = (nearest_expiry - sale_date).days
    if days_to_expiry ⩽ 3:
        log_warning(
            f"لات نزدیک به انقضا: {product_id}, "
            f"انقضا: {nearest_expiry} ({days_to_expiry} روز)"
        )

    # ۳. تخصیص به ترتیب FEFO
    allocations = []
    remaining = qty_needed
    for lot in lots:
        if remaining ⩽ 0:
            break
        take = min(lot.current_qty, remaining)
        allocations.append(AllocationItem(
            lot_id=lot.id,
            qty=take,
```

```
            unit_cost=lot.weighted_avg_cost,
            expiry_date=lot.expiry_date,
        ))
        remaining -= take

    if remaining > 0:
        raise InsufficientStockError(
            f"موجودی ناکافی: {qty_needed - remaining} عدد موجود، "
            f"{qty_needed} مورد نیاز"
        )

    return allocations
```

۶.۳ ردیابی بچ/لات در تمام جریان

تضمین یکپارچگی ردیابی لات در تمام چرخه زندگی محصول از تولید تا فروش، یکی از مهم‌ترین الزامات سیستم است. این ردیابی به‌گونه‌ای طراحی شده که در هر لحظه بتوان به این سوال پاسخ داد: «این کیک که در فروشگاه X فروخته شد، از کدام بچ تولید بود و با کدام بچ مواد اولیه ساخته شده بود؟»

برای پاسخ به این سوال، چندین مکانیزم به‌صورت هم‌زمان فعال هستند. هر بچ تولید، هنگام ثبت به یک لات جدید در `InventoryLots` متصل می‌شود. تمام حرکت‌های انبار (پخش، فروش، انتقال، برگشت) در جدول `InventoryMovements` با ارجاع به `lot_id` ثبت می‌شوند. در نهایت، تمام اسناد فروش و پخش در اقلام (`SalesLines`، `DispatchLines`) با `lot_id` ذخیره می‌شوند. این زنجیره ارجاع امکان Trace-back کامل را فراهم می‌کند.

> **مزیت Trace-back کامل**
>
> در صورت دریافت شکایت از یک فروشگاه درباره کیفیت یک کیک، سیستم در کمتر از ۱ ثانیه می‌تواند تمام فروشگاه‌هایی که همان بچ را دریافت کرده‌اند شناسایی کند و عامل پخش را برای جمع‌آوری فوری اعزام کند. این قابلیت در صنایع غذایی برای جلوگیری از آسیب به برند و سلامت مصرف‌کننده حیاتی است.

SECTION 07

مدل فروش امانی و قطعی

تفاوت انبار و حسابداری در دو مدل فروش

۷.۱ تعریف دو مدل

کسب‌وکار با دو مدل فروش موازی کار می‌کند که هر کدام منطق متفاوتی در انبار و حسابداری دارند. این دو مدل ممکن است برای یک فروشگاه واحد هم به‌صورت ترکیبی اعمال شوند (مثلاً ۸۰٪ امانی و ۲۰٪ قطعی برای محصولات خاص). سیستم باید بتواند این دو مدل را به‌صورت مجزا اما همزمان مدیریت کند:

- **فروش امانی (Consignment):** کارگاه کالا را به فروشگاه تحویل می‌دهد اما مالکیت کالا تا زمان فروش نهایی به مصرف‌کننده، در دست کارگاه می‌ماند. فروشگاه صرفاً به‌عنوان «نمایشگاه» عمل می‌کند. در این مدل، کالا در انبار مجازی فروشگاه (Warehouse type=virtual) ردیابی می‌شود و موجودی هنوز به کارگاه تعلق دارد.
- **فروش قطعی (Firm Sale):** کارگاه کالا را به فروشگاه می‌فروشد و مالکیت به فروشگاه منتقل می‌شود. کارگاه بابت کالا از فروشگاه طلبکار می‌شود. در این مدل، کالا از انبار کارگاه خارج شده و در تراز حسابداری به‌عنوان «درآمد فروش» و «بدهکار فروشگاه» ثبت می‌شود.

۷.۲ تفاوت در تراکنش‌های انبار و حسابداری

جدول زیر تفاوت دو مدل را در تمام جوانب نمایش می‌دهد. این تفاوت‌ها در Service Layer به‌صورت شفاف پیاده‌سازی شده‌اند و عامل پخش هنگام ثبت فروش، نوع را انتخاب می‌کند. سیستم بر اساس نوع انتخاب شده، فرایندهای متفاوتی را فعال می‌کند:

جنبه	مدل امانی (Consignment)	مدل قطعی (Firm Sale)
انتقال کالا	از انبار اصلی به انبار مجازی فروشگاه (نوع: transfer_out)	خروج مستقیم از انبار اصلی (نوع: out/sale)
مالکیت کالا	در دست کارگاه تا فروش نهایی	به فروشگاه منتقل می‌شود
سند حسابداری هنگام پخش	ثبت سند: «موجودی امانی فروشگاه» بدهکار، «موجودی انبار اصلی» بستانکار (تف Disposable)	ثبت سند: «حساب دریافتنی فروشگاه» بدهکار، «درآمد فروش + موجودی کالا» بستانکار
سند حسابداری هنگام فروش	ثبت سند: «درآمد فروش»، «بهای تمام‌شده فروش». خروج از انبار امانی.	هیچ سند جدیدی نیاز نیست (قبلاً در زمان پخش ثبت شده)
هزینه بازگشت	کالاهای برگشتی به انبار امانی برمی‌گردند (بدون تأثیر مالی)	کالاهای برگشتی به انبار اصلی برمی‌گردند و سند معکوس فروش صادر می‌شود
بهای تمام‌شده	در زمان فروش نهایی محاسبه می‌شود	در زمان پخش محاسبه می‌شود

> **تصمیم‌گیری در مورد مدل**
>
> انتخاب مدل امانی یا قطعی برای هر فروشگاه در تنظیمات فروشگاه (فیلد `sales_model` در جدول `Stores`) تعریف می‌شود. این فیلد می‌تواند در طول زمان تغییر کند (مثلاً وقتی یک فروشگاه اعتبار بسازد، از امانی به قطعی ارتقاء می‌یابد). تغییر در این فیلد نیازمند تأیید مدیر است و در Activity Log ثبت می‌شود.

اتصال انبار به حسابداری (Auto-Journal)

ثبت خودکار اسناد حسابداری دوطرفه از تراکنش‌های انبار

۸.۱ مکانیزم سیگنال‌محور

قلب یکپارچگی انبار با حسابداری، مکانیزم سیگنال‌های Django (Signals) است. هر Service انبار پس از commit موفق یک تراکنش (مثل ثبت فروش، ثبت تولید، ثبت انتقال)، یک سیگنال اختصاصی ارسال می‌کند. `Accounting Service` این سیگنال‌ها را دریافت کرده و بر اساس `JournalTemplate` مرتبط، یک سند حسابداری خودکار تولید می‌کند. این الگو جداسازی کامل بین لایه‌های انبار و حسابداری را تضمین می‌کند.

این طراحی مزیت مهمی دارد: Service انبار نیازی به دانستن جزئیات کدینگ حسابداری ندارد. اگر حسابدار تصمیم به تغییر کد حساب (مثلاً انتقال از ۱۰۱۰ به ۲۰۱۰) بگیرد، تنها کافی است `JournalTemplate` مربوطه را به‌روز کند و تمام تراکنش‌های بعدی از کد جدید استفاده خواهند کرد. این انعطاف در محیط کسب‌وکار واقعی که تغییرات کدینگ رایج است، ارزش قابل توجهی دارد.

۸.۲ کنترل توازن در سطح دیتابیس

تضمین توازن بدهکار/بستانکار نباید به لایه Application محدود باشد. در سطوح همزمانی بالا، ممکن است دو درخواست همزمان سندی را ثبت کنند که به‌صورت مجزا متوازن هستند اما در تجمیع نهایی اختلاف دارند. برای جلوگیری از این حالت، یک **Database Trigger** در PostgreSQL به‌صورت زیر پیاده‌سازی می‌شود:

```
-- PostgreSQL Trigger: کنترل توازن سند حسابداری
CREATE OR REPLACE FUNCTION check_journal_balance()
RETURNS TRIGGER AS
$$DECLARE v_total_debit NUMERIC(18,4); v_total_credit NUMERIC(18,4); v_diff NUMERIC$$
 LANGUAGE plpgsql;

CREATE TRIGGER trg_journal_balance
AFTER INSERT OR UPDATE OR DELETE ON journal_lines
FOR EACH ROW EXECUTE FUNCTION check_journal_balance();
```

۸.۳ سناریوی کامل: ثبت فروش و سند حسابداری

برای درک بهتر کل فرایند، سناریوی ثبت یک فروش امانی از ابتدا تا ثبت سند حسابداری را به‌صورت گام‌به‌گام مرور می‌کنیم. این سناریو نشان می‌دهد چگونه پنج لایه مختلف سیستم (API → Service → DB → Signal → Accounting) در یک فرایند واحد همکاری می‌کنند:

گام	لایه	عملیات
۱	API (DRF View)	دریافت POST /api/sales/ با JSON شامل type=consignment، store_id و لیست اقلام.
۲	Serializer	اعتبارسنجی: فروشگاه وجود دارد، نوع امانی مجاز است، موجودی کافی است.
۳	Sales Service	شروع تراکنش DB. فراخوانی allocate_lots_for_sale() برای FEFO. ساخت SalesHeader و SalesLines.
۴	Inventory Service	ثبت InventoryMovements (نوع: out) برای هر لات. کاهش current_qty در InventoryLots. به‌روزرسانی StockBalances.
۵	Sales Service	commit تراکنش. ارسال سیگنال post_save روی SalesHeader.
۶	Accounting Service	دریافت سیگنال. بازیابی JournalTemplate برای type=sale, sales_type=consignment. ساخت JournalHeader و JournalLines با کدهای مناسب.
۷	PostgreSQL Trigger	کنترل توازن هر خط. در صورت نامتوازن بودن، کل تراکنش rollback و خطا لاگ می‌شود.
۸	Notification Service	ارسال پیامک تأیید به عامل پخش و در صورت لزوم به فروشگاه.

SECTION 09

سیستم هشدار انتقال کالا (FEFO Transfer Suggester)

شناسایی و انتقال کالاهای نزدیک به انقضا از فروشگاه‌های کم‌فروش به پرفروش

۹.۱ مسئله و راه‌حل

یکی از منابع بزرگ ضایعات در صنعت کیک‌پزی، کالاهایی است که در فروشگاه‌های کم‌فروش باقی می‌مانند تا منقضی شوند، در حالی که همان محصول در فروشگاه‌های پرفروش نایاب می‌شود. این مشکل به دلیل نامتوازنی بین توزیع و تقاضای واقعی رخ می‌دهد. سیستم هشدار انتقال کالا با تحلیل روزانه داده‌های فروش و موجودی، این نامتوازنی‌ها را شناسایی کرده و انتقال کالا از فروشگاه کم‌فروش به پرفروش را پیشنهاد می‌دهد.

الگوریتم پیشنهاد انتقال به‌صورت یک Celery Task شبانه اجرا می‌شود (هر شب ساعت ۲۳:۰۰) و در پایان یک لیست پیشنهادات را در داشبورد مدیر نمایش می‌دهد. هر پیشنهاد شامل محصول، فروشگاه مبدأ (کم‌فروش)، فروشگاه مقصد (پرفروش)، تعداد کالا و لات مشخص است. مدیر پس از بازبینی، می‌تواند پیشنهادها را به تأیید برساند تا سند انتقال خودکار تولید شود.

۹.۲ شاخص‌ها و الگوریتم

الگوریتم بر اساس سه شاخص اصلی تصمیم‌گیری می‌کند. این شاخص‌ها به‌گونه‌ای طراحی شده‌اند که هم حساسیت به زمان انقضا و هم عملکرد فروشگاه را در نظر بگیرند، تا از انتقال بی‌مورد جلوگیری شود:

- **روزهای باقی‌مانده تا انقضا (DTE):** برای هر لات در هر فروشگاه، تعداد روز تا انقضا. لات‌هایی با DTE < ۵ روز در اولویت انتقال هستند.
- **نرخ فروش روزانه (DSR):** میانگین فروش روزانه این محصول در این فروشگاه در ۷ روز گذشته. این شاخص نشان می‌دهد آیا موجودی فعلی در این فروشگاه به‌اندازه کافی سریع فروخته می‌شود یا خیر.
- **Days of Stock (DOS):** نسبت موجودی فعلی به نرخ فروش روزانه. اگر DOS > DTE، یعنی موجودی فعلی قبل از انقضا به‌طور کامل فروخته نمی‌شود و مازاد وجود دارد.

```
# tasks/transfer_suggester.py
@shared_task
def generate_transfer_suggestions():
    """
    اجرای شبانه: شناسایی لات‌های نزدیک به انقضا در فروشگاه‌های کم‌فروش.
    """
    today = timezone.now().date()

    # تمام لات‌های فعال در انبارهای مجازی فروشگاه‌ها
    candidates = InventoryLot.objects.filter(
        warehouse__type='virtual',
        current_qty__gt=0,
        expiry_date__gte=today,
        expiry_date__lte=today + timedelta(days=7),
    ).select_related('product', 'warehouse__store')

    suggestions = []
    for lot in candidates:
        # محاسبه نرخ فروش روزانه ۷ روز گذشته
        dsr = calculate_dsr(lot.product_id, lot.warehouse.store_id, days=7)

        # Days of Stock
        dos = lot.current_qty / dsr if dsr > 0 else 999
        dte = (lot.expiry_date - today).days

        # شرط پیشنهاد: DOS > DTE (موجودی بیشتر از توان فروش است)
        if dos > dte:
            # جستجوی فروشگاه پرفروش نزدیک با موجودی کم
            dest_store = find_high_demand_store(
                product_id=lot.product_id,
                exclude_store_id=lot.warehouse.store_id,
                max_distance_km=50,
            )
            if dest_store:
                # محاسبه تعداد قابل انتقال
                qty_to_transfer = min(
                    lot.current_qty - (dsr * dte),  # مازاد بر نیاز فروشگاه فعلی
                    dest_store.daily_demand(lot.product_id) * 3,  # حداکثر ۳ روز
نیاز مقصد
                )
                if qty_to_transfer > 5:  # حداقل ۵ عدد منطقی است
                    suggestions.append(TransferSuggestion(
                        lot_id=lot.id,
                        from_store_id=lot.warehouse.store_id,
                        to_store_id=dest_store.id,
                        qty=qty_to_transfer,
                        urgency='high' if dte <= 3 else 'medium',
                        score=calculate_priority_score(dte, dos, qty_to_transfer),
                    ))
```

```
    # ذخیره پیشنهادات در دیتابیس
    TransferSuggestion.objects.bulk_create(suggestions)

    # ارسال پیامک اطلاع به مدیر
    if suggestions:
        send_sms(
            to=settings.MANAGER_PHONE,
            template='transfer_alert',
            context={'count': len(suggestions)},
        )

    return len(suggestions)
```

مزیت محاسباتی

بر اساس تخمین محافظه‌کارانه، اگر این سیستم منجر به کاهش ۱۵٪ ضایعات شود، در مقیاس سالانه با ۳۰ محصول و ۲۰۰ فروشگاه، صرفه‌جویی مالی قابل توجهی ایجاد می‌کند. این یکی از مهم‌ترین ROIهای پروژه است و باید در گزارش‌های مالی فاز ۶ به‌طور کمی اندازه‌گیری شود.

SECTION 10

سیستم پیامکی و یادآوری سرکشی

زمان‌بندی، قالب‌بندی و ارسال پیامک خودکار به عوامل پخش

۱۰.۱ معماری سرویس اعلان

سرویس اعلان به‌صورت یک Celery Worker مستقل پیاده‌سازی شده که صف ارسال پیامک را از جدول SmsQueue پردازش می‌کند. این جدول توسط سایر Serviceها (مانند Transfer، Inventory، Sales) به‌صورت ناهمزمان پر می‌شود. سرویس هر ۳۰ ثانیه صف را پایش می‌کند و پیامک‌های آماده ارسال را به پروایدر (کاوه‌نگار) تحویل می‌دهد.

این جداسازی مزیت مهمی دارد: هیچ Serviceای مستقیماً با API پیامک درگیر نمی‌شود. اگر پروایدر پیامک قطع شود، صف در دیتابیس رشد می‌کند و پس از بازگشت سرویس، تمام پیامک‌های معوق به‌صورت خودکار ارسال می‌شوند. این تضمین می‌کند که هیچ پیامکی (به‌ویژه یادآوری سرکشی بحرانی) از دست نرود.

۱۰.۲ قالب‌های پیامک و یادآوری سرکشی

قالب‌های پیامک در جدول SmsTemplate با placeholderهای Jinja2 ذخیره می‌شوند. این طراحی اجازه می‌دهد قالب‌ها بدون تغییر کد، از طریق Admin Django ویرایش شوند. جدول زیر قالب‌های اصلی مورد نیاز را نشان می‌دهد:

نام قالب	دریافت‌کننده	متن (با Placeholder)
`visit_reminder`	عامل پخش	«{{distributor_name}} عزیز، یادآوری سرکشی روز {{date}}: {{store_count}} فروشگاه. اولین فروشگاه: {{first_store}} ساعت {{first_time}}. لطفاً به موقع حاضر شوید.»
`sale_confirmation`	فروشگاه	«{{store_name}}، فروش به مبلغ {{total_amount}} ریال با شماره {{sale_no}} ثبت شد. اقلام: {{items_summary}}.»
`expiry_alert`	مدیر	«هشدار: {{count}} لات نزدیک به انقضا. اولویت: {{highest_priority_lot}}. وارد داشبورد شوید.»
`transfer_request`	فروشگاه مقصد	«{{from_store}} درخواست انتقال {{qty}} عدد {{product}} کرده. کد انتقال: {{transfer_no}}. لطفاً تأیید کنید.»
`low_stock`	انباردار	«موجودی {{product}} در {{warehouse}} به {{qty}} رسید (زیر حداقل {{min_qty}}).»
`journal_error`	حسابدار	«خطای توازن سند {{journal_no}}: {{diff_amount}} ریال. لطفاً بررسی کنید.»

۱۰.۳ یکپارچگی با کاوه‌نگار

پروایدر کاوه‌نگار به‌دلیل پوشش گسترده و پایداری بالا در ایران انتخاب شده است. این سرویس یک REST API ساده با احراز هویت API Key ارائه می‌دهد. کد زیر نمایش پیاده‌سازی کلاینت آن است که از طریق یک Provider Abstraction در `SMSProviderInterface` کپسوله شده تا در صورت نیاز به تعویض پروایدر (به فراپیامک)، تنها یک کلاس تغییر کند:

```
# notifications/providers/kavenegar.py
import requests
from django.conf import settings
from .base import SMSProviderInterface

class KavenegarProvider(SMSProviderInterface):
    API_URL = "https://api.kavenegar.com/v1/{key}/verify/lookup.json"

    def send(self, to: str, template: str, params: dict) → dict:
        url = self.API_URL.format(key=settings.KAVENEGAR_API_KEY)
        response = requests.post(url, data={
            'receptor': to,
            'token': params.get('token', ''),
            'token2': params.get('token2', ''),
            'token3': params.get('token3', ''),
            'template': template,
        }, timeout=10)
        response.raise_for_status()
        return response.json()

    def send_batch(self, messages: list) → list:
        """ارسال گروهی برای کاهش هزینه."""
        # کاوه‌نگار از ارسال گروهی پشتیبانی می‌کند
        results = []
        for msg in messages:
            try:
                result = self.send(msg['to'], msg['template'], msg['params'])
                results.append({'status': 'success', 'msg_id':
result['messageid']})
            except Exception as e:
                results.append({'status': 'failed', 'error': str(e)})
                # Retry در صف با Exponential Backoff
                schedule_retry(msg, attempt=1)
        return results
```

استراتژی Retry

در صورت خطای ارسال (شبکه، سرویس‌ناپذیر)، پیامک با Exponential Backoff تلاش مجدد می‌کند: ۱ دقیقه، ۵ دقیقه، ۱۵ دقیقه، ۱ ساعت. پس از ۴ تلاش ناموفق، پیامک به‌عنوان «Failed» علامت‌گذاری و به مدیر اطلاع داده می‌شود.

فرانت‌اند و تجربه کاربری (RTL + شمسی)

Next.js + TypeScript + Tailwind RTL + تقویم جلالی

۱۱.۱ استک فرانت‌اند

فرانت‌اند با Next.js 14 (App Router) و TypeScript توسعه داده می‌شود. انتخاب Next.js به دلیل سه مزیت اصلی است: (۱) پشتیبانی SSR برای بهبود LCP در صفحات کلیدی مثل داشبورد و گزارش‌ها، (۲) PWA برای دسترسی آفلاین عوامل پخش در مناطق با اینترنت ناپایدار، و (۳) Code Splitting خودکار برای کاهش bundle size.

Tailwind CSS با RTL Plugin برای استایل‌بندی انتخاب شده زیرا این پکیج با ابزار `tailwindcss-rtl` به‌طور خودکار direction-aware utility classes تولید می‌کند. برای مثال، `pl-4` در حالت RTL به‌طور خودکار به padding-right تبدیل می‌شود. این مزیت توسعه سریع بدون نیاز به نوشتن CSS سفارشی را فراهم می‌کند.

+۹۰	۱۵۰ms	PWA
هدف Lighthouse	هدف FCP	دسترسی آفلاین

۱۱.۲ فونت و RTL

فونت Vazirmatn به‌عنوان فونت اصلی فارسی انتخاب شده است. این فونت به‌دلیل وزن‌های کامل (۳۰۰ تا ۹۰۰)، پشتیبانی از اعداد فارسی و قالب‌بندی صحیح کاراکترهای RTL، بهترین انتخاب برای پروژه‌های مدرن فارسی است. فونت از Google Fonts CDN بارگذاری می‌شود با `font-display: swap` برای جلوگیری از FOIT (Flash of Invisible Text).

تمام صفحات با `dir="rtl"` و `lang="fa"` در تگ html شروع می‌شوند. تقویم جلالی با کتابخانه `jalaali-js` برای تبدیل میلادی-شمسی و کامپوننت `react-jalaali-datepicker` برای رابط کاربری Date Picker استفاده می‌شود. تمام نمایش‌های تاریخ در فرانت‌اند به شمسی تبدیل می‌شوند، در حالی که در بک‌اند و دیتابیس به‌صورت میلادی ذخیره می‌گردند.

۱۱.۳ نقش‌های کاربری و صفحات

سیستم چهار نقش کاربری دارد که هر کدام مجموعه‌ای از صفحات اختصاصی دارند. رابط کاربری با الگوی Master-Detail طراحی شده: یک لیست در سمت راست و جزئیات در سمت چپ (در حالت RTL). جدول زیر نقش‌ها و صفحات کلیدی آنها را نشان می‌دهد:

نقش	صفحات کلیدی
مدیر	داشبورد بلادرنگ، گزارش‌های فروش/ضایعات/سود، تأیید انتقال FEFO، تنظیمات کاربران و نقش‌ها، تنظیم قالب‌های پیامک.
حسابدار	دفتر روزنامه، تراز آزمایشی، کدینگ حساب‌ها، کنترل توازن روزانه، گزارش سود و زیان، صورت‌های مالی.
انباردار	ثبت تولید، ورود/خروج کالا، موجودی لحظه‌ای انبارها، درخواست انتقال، گزارش ضایعات.
عامل پخش	سبد پخش روزانه (با اپ موبایل PWA)، ثبت فروش امانی/قطعی، ثبت برگشت، مشاهده موجودی فروشگاه، ثبت موقعیت GPS سرکشی.

۱۱.۴ اپلیکیشن موبایل PWA برای عوامل

عوامل پخش در مسیر حرکت هستند و اغلب در مناطقی با اینترنت ناپایدار کار می‌کنند. برای پشتیبانی از این حالت، یک PWA با Service Worker توسعه داده می‌شود که امکان ثبت فروش آفلاین را فراهم می‌کند. وقتی اینترنت قطع است، فروش‌ها در IndexedDB ذخیره و پس از بازگشت اینترنت، با Celery Beat به‌صورت خودکار با سرور همگام می‌شوند.

> **تضاد FEFO با حالت آفلاین**
>
> در حالت آفلاین، عامل نمی‌تواند آخرین وضعیت موجودی را ببیند که ممکن است منجر به ثبت فروش کالای ناموجود شود. راه‌حل: هنگام شروع روز، عامل آخرین وضعیت موجودی هر فروشگاه را در IndexedDB دانلود می‌کند و در طول روز فقط با این snapshot کار می‌کند. وقتی اینترنت بازگشت، اگر اختلافی باشد (مثلاً فروشگاه از منبع دیگری کالا خریده باشد)، خطا به مدیر اطلاع داده می‌شود و عامل باید بازبینی کند.

SECTION 12

نقشه راه پروژه (Roadmap)

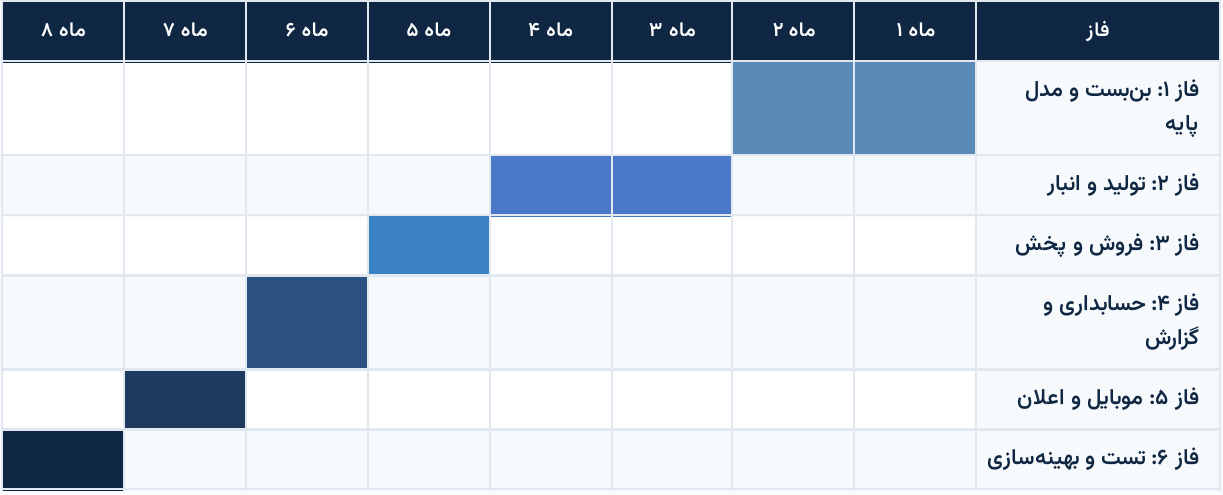
شش فاز هشت‌ماهه با تحویل تدریجی MVP

۱۲.۱ اصول Roadmap

نقشه راه پروژه بر اساس دو اصل طراحی شده: (۱) هر فاز یک MVP قابل دمو تحویل می‌دهد که به کسب‌وکار اجازه می‌دهد زودتر از موعد نهایی ارزش دریافت کند. (۲) ریسک‌های فنی بزرگ (مثل FEFO و میانگین موزون) در فازهای اولیه قرار گرفته‌اند تا اگر مشکلی پیش بیاید، زمان کافی برای جبران وجود داشته باشد. این دو اصل احتمال موفقیت پروژه را به‌طور قابل توجهی افزایش می‌دهند.

۱۲.۲ نمودار گانت فازها

نمودار گانت زیر نشان‌دهنده تایم‌لاین شش فاز در هشت ماه است. هر فاز بین ۱ تا ۲ ماه طول می‌کشد و در پایان هر فاز یک Demoo Day برگزار می‌شود که تیم توسعه و کسب‌وکار نتایج را بازبینی می‌کنند. پس از فاز ۴، یک Soft Launch به ۱۰٪ فروشگاه‌ها انجام می‌شود و در فاز ۶ Rollout کامل رخ می‌دهد.



۱۲.۳ جزئیات فازها

هر فاز شامل مجموعه‌ای از قابلیت‌هاست که باید به‌طور کامل پیاده‌سازی شوند تا فاز قابل دمو باشد. جدول زیر قابلیت‌های کلیدی هر فاز را نشان می‌دهد:

فاز	مدت	قابلیت‌های کلیدی	Deliverable فاز
۱	۲ ماه	راه‌اندازی زیرساخت (Docker، CI/CD)، پیاده‌سازی ۱۱ App پایه با Migration، احراز هویت JWT، RBAC، ادمین Django سفارشی، تعریف اولیه ۳۰ محصول.	MVP با امکان ثبت کاربر، ورود، تعریف محصول، نمایش لیست انبارها.
۲	۲ ماه	پیاده‌سازی کامل Inventory Service با FEFO، Production Service با میانگین موزون، API‌های REST، PostgreSQL Triggers، اولین گزارش‌های موجودی.	MVP با امکان ثبت بچ تولید، مشاهده لات‌ها، ثبت حرکت انبار.
۳	۱.۵ ماه	Sales Service (امانی/قطعی)، Dispatch Service، Return Service، فرانت‌اند Next.js با صفحات فروش و پخش، اولین نسخه اپ موبایل (Read-Only).	MVP با امکان ثبت کامل چرخه فروش امانی و قطعی.
۴	۱ ماه	Accounting Service با JournalTemplates، Auto-Journal برای Django Signals، گزارش‌های مالی پایه، داشبورد مدیر.	MVP با یکپارچگی کامل انبار-حسابداری. Soft Launch به ۲۰ فروشگاه.
۵	۱ ماه	Notification Service با کاوه‌نگار، Celery Beat برای یادآوری سرکشی، Transfer Suggester، PWA Mobile برای عوامل با حالت آفلاین.	MVP با سیستم پیامکی کامل و اپ موبایل فعال.
۶	۰.۵ ماه	تست بار (Locust)، بهینه‌سازی کوئری‌ها، پیاده‌سازی Sentry و Grafana، بکاپ روزانه، مستندات کاربران، آموزش تیم کسب‌وکار.	نسخه ۱.۰ نهایی، آماده Rollout کامل به ۲۰۰ فروشگاه.

استراتژی تست و DevOps

CI/CD، تست‌های خودکار، مانیتورینگ و بکاپ

۱۳.۱ هرم تست‌ها

سیستم تست بر اساس هرم کلاسیک تست نرم‌افزار طراحی شده که شامل سه لایه است. این هرم تضمین می‌کند که ۸۰٪ کد با تست‌های سریاع واحد پوشش داده شود، ۱۵٪ با تست‌های یکپارچگی برای بررسی تعامل بین Serviceها، و ۵٪ با تست‌های End-to-End برای سناریوهای کلیدی کسب‌وکار. این ترکیب سرعت توسعه و اطمینان از صحت عملکرد را هم‌زمان تأمین می‌کند:

- **Unit Tests (۸۰٪):** تست هر تابع Service به‌صورت مستقل با mocking پایگاه داده. ابزار: pytest + pytest-django + factory_boy. هدف P95 زیر ۳۰ ثانیه برای کل suite.
- **Integration Tests (۱۵٪):** تست تعامل چند Service با دیتابیس واقعی (PostgreSQL در Docker). ابزار: pytest-django + testfixtures. این لایه به‌طور خاص FEFO و میانگین موزون را با داده‌های واقعی تست می‌کند.
- **E2E Tests (۵٪):** تست سناریوهای کامل کسب‌وکار از طریق API. ابزار: pytest + requests. سناریوها: ثبت تولید → پخش → فروش امانی → تسویه روزانه → کنترل توازن حسابداری.

۱۳.۲ CI/CD با GitHub Actions

یک Pipeline کامل CI/CD با GitHub Actions پیاده‌سازی می‌شود که در هر Push یا Pull Request به‌طور خودکار اجرا می‌شود. این Pipeline شامل چهار stage است که به ترتیب اجرا می‌شوند. اگر هر stage ناموفق باشد، Pipeline متوقف و به تیم اطلاع داده می‌شود. این تضمین می‌کند که هیچ کد نامعتبری به شاخه اصلی ادغام نشود:

```yaml
# .github/workflows/ci.yml
name: CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install black flake8 mypy pylint
      - run: black --check .
      - run: flake8 .
      - run: mypy --strict backend/
      - run: pylint --fail-under=8 backend/

  test:
    needs: lint
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ['5432:5432']
      redis:
        image: redis:7
        ports: ['6379:6379']
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest --cov=backend --cov-report=xml --cov-fail-under=80

  security:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: pip install bandit safety
      - run: bandit -r backend/ -ll
      - run: safety check

  deploy-staging:
    needs: [lint, test, security]
    if: github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
```

```
    steps:
      - run: docker build -t registry/bakery:github.sha. — run :
dockerpushregistry/bakery :{{ github.sha }}
      - run: ssh deploy@staging "cd /opt/bakery && docker-compose pull && docker-
compose up -d"
```

۱۳.۳ مانیتورینگ و بکاپ

سیستم مانیتورینگ شامل سه لایه است. **Sentry** برای ردیابی خطاهای کد در زمان واقعی (Frontend و Backend) با یکپارچگی کامل با Django و Next.js. **Grafana + Prometheus** برای Metrics زیرساختی شامل CPU، RAM، Disk I/O، تعداد درخواست‌ها، Response Time توزیع‌شده، و تعداد تراکنش‌های DB. **Loki + Promtail** برای تجمیع لاگ‌های Application.

بکاپ روزانه پایگاه داده با pg_dump انجام می‌شود و فایل‌های بکاپ به یک Object Storage (MinIO یا S3) منتقل می‌شوند. سیاست Retention شامل نگه‌داری ۷ بکاپ روزانه، ۴ هفتگی، و ۱۲ ماهانه است که مجموعاً ۲۳ بکاپ را در هر لحظه نگه‌داری می‌کند. علاوه بر این، یک Replication Stream به سرور Standby در دیتاسنتر دوم برای Disaster Recovery فعال است که RPO را به زیر ۵ دقیقه می‌رساند.

> **Health Check آسان**
>
> یک endpoint ساده `/api/health/` شامل وضعیت تمام وابستگی‌ها (DB، Redis، RabbitMQ، Celery) را برمی‌گرداند. این endpoint توسط UptimeRobot هر ۱ دقیقه پایش می‌شود و در صورت قطعی، تیم Ops در عرض ۵ دقیقه از طریق تلگرام آگاه می‌شود.

SECTION 14

تحلیل ریسک و استراتژی کاهش

شناسایی، اولویت‌بندی و راهکار مدیریت ریسک‌های پروژه

۱۴.۱ متدولوژی تحلیل ریسک

تحلیل ریسک بر اساس ماتریس متداول احتمال × اثر انجام شده است. هر ریسک بر اساس دو محور احتمال وقوع (۱-۵) و شدت اثر (۱-۵) ارزیابی می‌شود. امتیاز نهایی (ضرب دو محور) تعیین‌کننده اولویت توجه است: امتیاز بالای ۱۵ به‌عنوان بحرانی، ۱۰-۱۵ به‌عنوان بالا، ۵-۹ به‌عنوان متوسط، و زیر ۵ به‌عنوان پذیرفته‌شده در نظر گرفته می‌شود. این متدولوژی به تیم اجازه می‌دهد منابع را بر اساس اولویت واقعی تخصیص دهد.

۱۴.۲ جدول ریسک‌های اصلی

پس از بررسی کامل پروژه، پنج ریسک اصلی شناسایی شده‌اند. برای هر ریسک، راهکار کاهش مشخص و اقدام واکنشی در صورت وقوع تعریف شده است. این جدول باید در طول پروژه توسط Project Manager به‌روزرسانی شود و در پایان هر فاز بازبینی گردد:

#	ریسک	احتمال	اثر	امتیاز	راهکار کاهش (Mitigation)	اقدام واکنشی (Contingency)
۱	پیچیدگی FEFO و میانگین موزون در حجم بالا	۳	۵	۱۵	پیاده‌سازی PostgreSQL Functions با Row-Level Locking. تست بار زودهنگام در فاز ۲.	مهاجرت به Celery Task ناهمزمان برای محاسبات سنگین. ارتقاء به PostgreSQL Partitioning.
۲	عدم تطابق داده‌های قدیمی با ساختار جدید	۴	۳	۱۲	اسکریپت Migration با Data Validation. اجرای Dry-Run در محیط Staging.	ورود دستی داده‌های پایه با تأیید مدیر. نگه‌داری سیستم قدیمی به مدت ۳ ماه موازی.
۳	تاخیر تیم توسعه در فازهای حساس	۳	۴	۱۲	تعریف Milestone هفتگی با Demo. اگر فاز ۲ به تأخیر بیفتد، فاز ۶ حذف می‌شود و ۵ گسترش می‌یابد.	استفاده از منابع Outsource برای ۲ هفته بحرانی. کاهش Scope فازهای آخر.
۴	قطعی سرویس پیامک یا تغییر API	۳	۳	۹	پیاده‌سازی SMSProviderInterface برای تعویض سریع. Reserve Provider دوم (فراپیامک).	فعال‌سازی فراپیامک در کمتر از ۱۰ دقیقه. اطلاع‌رسانی مدیر از طریق تلگرام.
۵	مشکلات پایداری در Soft Launch	۳	۴	۱۲	Soft Launch به ۲۰ فروشگاه (نه ۲۰۰). مانیتورینگ شدید با Sentry و Grafana.	قابلیت Rollback به نسخه قبلی با Docker. نگه‌داری سیستم قدیمی ۱ ماه موازی.

۱۴.۳ تخمین هزینه

تخمین هزینه‌ها بر اساس تجربه بازار ایران در سال ۱۴۰۴ انجام شده است. این ارقام شامل هزینه‌های زیرساخت، تیم توسعه و لایسنس‌ها برای یک چرخه ۸ ماهه پروژه و ۱۲ ماه پشتیبانی پس از تحویل است. تمام ارقام به ریال ایران است:

عنوان هزینه	ماهانه	کل (۲۰ ماه)
تیم توسعه (۵ نفر × میانگین دستمزد ارشد)	۲٬۸۰۰٬۰۰۰٬۰۰۰	۴۸٬۰۰۰٬۰۰۰٬۰۰۰
VPS اختصاصی (۱۶GB RAM، ۴ Core)	۱۲۰٬۰۰۰٬۰۰۰	۲٬۴۰۰٬۰۰۰٬۰۰۰
Object Storage و Bandwidth	۳۰٬۰۰۰٬۰۰۰	۶۰۰٬۰۰۰٬۰۰۰
پروایدر پیامک (هر ماه حدود ۵۰هزار پیامک)	۹۰٬۰۰۰٬۰۰۰	۱٬۸۰۰٬۰۰۰٬۰۰۰
دامنه، SSL، Backup Storage	۱۵٬۰۰۰٬۰۰۰	۳۰۰٬۰۰۰٬۰۰۰
لایسنس Sentry Pro و ابزارهای مانیتورینگ	۲۰٬۰۰۰٬۰۰۰	۴۰۰٬۰۰۰٬۰۰۰
جمع	**۳٬۰۵۵٬۰۰۰٬۰۰۰**	**~۵۳٫۵ میلیارد ریال**

ROI پروژه

اگر سیستم تنها از طریق کاهش ۱۵٪ ضایعات و کاهش ۳۰٪ هزینه‌های عملیاتی پخش (سرکشی بهینه‌تر، خطاهای ثبت کمتر) عمل کند، بازگشت سرمایه (ROI) در حدود ۱۴-۱۸ ماه پس از Rollout کامل محقق می‌شود. این محاسبه بر اساس حجم فعلی کسب‌وکار (۲۰۰ فروشگاه و ۳۰ محصول) انجام شده.

SECTION 15

جمع‌بندی و گام‌های بعدی

پایان سند و توصیه‌های اجرایی

این سند معماری یک نقشه راه کامل و قابل اجرا برای جایگزینی سیستم شکننده فعلی با یک پلتفرم پایدار، چندکاربره و تحت وب ارائه می‌دهد. طراحی بر اساس تجربه صنعتی و استانداردهای مهندسی نرم‌افزار مدرن انجام شده و تمام الزامات کسب‌وکار (FEFO، میانگین موزون، حسابداری دوطرفه، RTL، تقویم شمسی، چندکاربره) را در یک معماری یکپارچه پوشش می‌دهد.

چهار توصیه اجرایی برای موفقیت پروژه ارائه می‌شود. اول، فاز صفر (یک هفته قبل از شروع فاز ۱) را به تأیید نهایی فرضیات، تهیه زیرساخت اولیه و تیم‌سازی اختصاص دهید. دوم، در پایان هر فاز، یک جلسه بازبینی با کسب‌وکار برگزار کنید تا بازخورد زنده دریافت شود و در صورت لزوم Small Scope Adjustments انجام گیرد. سوم، از فاز ۲ به بعد، یک کارشناس حسابداری ارشد کسب‌وکار را به‌عنوان مشاور فنی در فرایند توسعه دخالت دهید تا الزامات مالی به‌درستی پیاده‌سازی شوند. چهارم، پس از تحویل نسخه ۱.۰، یک دوره پشتیبانی ۱۲ ماهه با تیم توسعه برای رفع Bug و اضافه کردن قابلیت‌های کوچک پیش‌بینی کنید.

این سند به‌عنوان نقطه شروع کار است و در طول فاز ۱ با جزئیات بیشتر در زمینه API Spec، UI Wireframes و Database Migration Script تکمیل خواهد شد. هرگونه تغییر در فرضیات یا اهداف کسب‌وکار نیازمند بازنگری این سند است و باید از طریق کنترل نسخه مدیریت گردد. با پیاده‌سازی این نقشه راه، کسب‌وکار در پایان ۸ ماه

یک سیستم نرم‌افزاری پایدار و قابل توسعه خواهد داشت که برای سال‌های آتی نیازهای عملیاتی و مالی را پوشش می‌دهد.

آماده توسعه

این سند نقطه شروع پروژه است. در پایان فاز ۱، نسخه ۲.۰ این سند با جزئیات API، Wireframes و Migration Scriptها منتشر خواهد شد.

NEXT STEPS

- تأیید فرضیات با ذی‌نفعان
- تشکیل تیم توسعه
- تهیه زیرساخت اولیه
- شروع فاز ۱ (بن‌بست)

CONTACT

معمار سیستم

تیم ارشد مهندسی نرم‌افزار

نسخه سند: ۱.۰

تاریخ: ۱۴۰۴